2.2 GitHub Copilot w praktyce: Wykorzystanie czatu, generowanie testów jednostkowych i refaktoryzacja

Moduł 2: Ekosystem Vibe Coding i AI-first IDE

Krzysztof Rytlewski · 2026

GitHub Copilot – architektura i tryby pracy

GitHub Copilot to asystent AI zintegrowany z edytorem (VS Code, JetBrains, Neovim), oparty na modelach OpenAI Codex/GPT-4o. Funkcjonuje jako rozszerzenie do istniejącego edytora – łatwiejszy w adopcji bez zmiany środowiska.

Tryb	Skrót	Zastosowanie
Inline suggestions	automatyczne	Uzupełnianie kodu (ghost text)
Copilot Chat	`Ctrl+Alt+I`	Rozmowa w panelu bocznym
Inline Chat	`Ctrl+I`	Szybka edycja w miejscu kursora
Quick Chat	`Ctrl+Shift+Alt+L`	Jednorazowe pytanie
@workspace	w czacie	Pytanie z kontekstem całego projektu

Jak działa dopasowanie sugestii inline

Copilot wysyła do modelu aktualny plik, kilka ostatnio edytowanych plików oraz komentarze/nagłówki jako kontekst. Im bardziej opisowy komentarz poprzedzający kod, tym trafniejsza propozycja.

```
// Pobiera użytkownika z MongoDB po ID,
// rzuca NotFoundError jeśli nie istnieje
async function getUserById(id) {
  // Copilot zasugeruje kod zgodny z powyższym opisem
}
```

Precyzyjny opis intencji = lepsza sugestia. To prosta, ale kluczowa zasada efektywnej pracy z Copilotem.

Uczestnicy kontekstowi (`@`) w Copilot Chat

Uczestnicy rozszerzają kontekst zapytania o konkretne źródła informacji:

- `@workspace` – przeszukuje cały projekt, dołącza relevantne pliki. Niezbędny przy pytaniach architekturalnych: `@workspace jak jest zorganizowana warstwa serwisów?`
- `@vscode` – odpowiada na pytania o edytor, ustawienia i skróty klawiszowe.
- `@terminal` – analizuje ostatnie polecenie i błąd w terminalu, proponuje poprawkę.
- `@github` – (Copilot Enterprise) dostęp do repozytoriów, PR i Issues na GitHub.

Komendy ukośnikowe (`/`) w Copilot Chat

Komenda	Opis
`/explain`	Wyjaśnia zaznaczony kod lub cały plik
`/fix`	Naprawia błąd (najlepiej z zaznaczonym fragmentem)
`/tests`	Generuje testy jednostkowe dla zaznaczonej funkcji
`/doc`	Dodaje dokumentację JSDoc/TSDoc do zaznaczonego kodu
`/simplify`	Upraszcza zaznaczony fragment
`/new`	Tworzy nowy plik lub projekt na podstawie opisu

Przykład: gdy w terminalu pojawia się błąd modułu, `@terminal /fix` – Copilot analizuje błąd i wskazuje brakujący plik lub błędną ścieżkę importu.

Workflow generowania testów z `/tests`

Komenda `/tests` analizuje sygnaturę i implementację funkcji, identyfikuje ścieżki wykonania i generuje gotowe testy.

1. Zaznacz funkcję lub cały plik (`Ctrl+A` dla całego pliku).
2. Napisz `/tests` (opcjonalnie: `/tests używaj Jest, mockuj moduł mongoose`).
3. Przejrzyj wygenerowane testy – sprawdź pokrycie kluczowych przypadków.
4. Kliknij *Insert into new file* lub skopiuj do pliku testowego.
5. Uruchom testy i popraw ewentualne importy lub mocki.

Przykład – generowanie testów dla serwisu Node.js

Copilot generuje m.in. happy path i przypadek błędu dla funkcji `getUserById`:

```
describe('getUserById', () => {
  it('zwraca użytkownika gdy istnieje', async () => {
    const mockUser = { _id: '123', name: 'Jan Kowalski' };
    User.findById.mockResolvedValue(mockUser);
    const result = await getUserById('123');
    expect(result).toEqual(mockUser);
  });

  it('rzuca błąd gdy użytkownik nie istnieje', async () => {
    User.findById.mockResolvedValue(null);
    await expect(getUserById('999')).rejects.toThrow('User 999 not found');
  });
});
```

Wskazówki jakości przy generowaniu testów

Domyślnie Copilot generuje solidny zestaw testów, ale warto doprecyzować kilka kwestii:

- Podaj informację o frameworku, żeby uniknąć mieszania składni Mocha i Jest: `/tests używaj Jest`.
- Jeśli Copilot nie mockuje zewnętrznych zależności, doprecyzuj: `/tests zamockuj wszystkie połączenia z bazą danych`.
- Zawsze weryfikuj, czy testy faktycznie testują **logikę biznesową**, a nie tylko sprawdzają wywołania metod.
- Sprawdź, czy mocki są prawidłowo inicjalizowane i czyszczone między testami.

Refaktoryzacja wspomagana przez Copilot

Copilot Chat jest szczególnie przydatny przy refaktoryzacji, ponieważ uwzględnia kontekst całego pliku i proponuje zmiany zgodne z istniejącym stylem kodu.

Typowe scenariusze:

- **Ekstrakcja funkcji** – `wyodrębnij tę logikę do osobnej funkcji o nazwie validatePayload`.
- **Migracja składni** – `przekonwertuj wszystkie .then().catch() na async/await`.
- **Zwiększenie czytelności** – `zmień warunki zagnieżdżone na early return`.
- **Poprawa typowania TypeScript** – `dodaj ścisłe typy do parametrów i wartości zwracanych`.

Przykład refaktoryzacji Express route handlera

Prompt: *„Wyodrębnij logikę biznesową do funkcji `createUser` w osobnym serwisie. Handler powinien tylko wywoływać serwis i mapować wyniki na odpowiedzi HTTP."*

Po refaktoryzacji logika biznesowa trafia do `user.service.js` i staje się **testowalną jednostką niezależną od HTTP**. Handler jest cienką warstwą mapującą wyniki serwisu na statusy i JSON.

Dla zmian obejmujących wiele plików: `@workspace znajdź wszystkie miejsca gdzie używamy callback-based API mongoose i zaproponuj migrację na async/await`. Copilot wskazuje pliki i pokazuje przykłady zmian.

Plik `.github/copilot-instructions.md`

Odpowiednik `.cursorrules` – stały system prompt dołączany automatycznie do każdego zapytania w Copilot Chat dla całego repozytorium.

```
## Projekt
Backend: Node.js 20 + Express 4 + MongoDB 7 (Mongoose 8).
Frontend: Angular 17 (standalone components, Angular Signals).

## Standardy kodu
- Typescript strict mode – zawsze typuj zwracane wartości
- Async/await zamiast Promise chains
- Nazwy klas: PascalCase; pliki: kebab-case.ts

## Testy (Jest / Karma+Jasmine)
- Mockuj zewnętrzne zależności
- Każda funkcja publiczna: co najmniej 2 testy
```

Copilot w pracy zespołowej – code review

Rozszerzenie GitHub Pull Requests + Copilot umożliwia asystowanie przy przeglądach bezpośrednio w VS Code:

- **Generowanie opisu PR** – na podstawie dołączonych commitów (`@github /pr` lub przycisk w widoku PR).

- **Wyjaśnianie diffa** – zaznacz zmienione linie i zapytaj `wyjaśnij tę zmianę i jej wpływ na resztę systemu`.

- **Sugestie ulepszeń** – `@workspace sprawdź ten diff pod kątem potencjalnych problemów z wydajnością lub bezpieczeństwem`.

Copilot przyspiesza przegląd kodu bez zastępowania osądu ludzkiego recenzenta.